

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 7

End-to-End Study of Autoencoders, Convolutional Autoencoders, Denoising Autoencoders and Variational Autoencoders

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Name: T. A. Prasanna

Roll No.: 24110124

Registration No.: 24011101113

Github: <https://github.com/Prasannnnnnnnnaaaaa/Deep-Learning-Lab.git>

1. Objective

The objective of this experiment is to develop an end-to-end understanding of autoencoders and their variants for image representation, reconstruction, denoising and generative modeling.

The experiment begins with a fully connected autoencoder, progresses to a Convolutional Autoencoder (CAE), introduces image corruption for denoising, and finally develops a Variational Autoencoder (VAE). Students will visualize the learned latent representation, compare reconstruction quality using quantitative and qualitative measures, and explore the generative capability of the VAE.

The same image dataset and a controlled experimental protocol should be used throughout the experiment wherever possible.

2. Learning Outcomes

After completing this experiment, students will be able to:

- explain the encoder–latent–decoder structure of an autoencoder;
- prepare image data for reconstruction-based learning;
- implement a fully connected autoencoder;
- implement a Convolutional Autoencoder for image reconstruction;
- introduce controlled noise and build a denoising autoencoder;
- evaluate reconstruction using MSE, MAE and SSIM;
- visualize and interpret latent representations;
- explain the probabilistic latent representation of a VAE;
- implement the reparameterization trick;
- calculate and interpret reconstruction and KL-divergence losses;
- generate new images by sampling from a learned latent space; and
- compare deterministic and variational latent representations.

3. Dataset and Experimental Setup

Dataset: MNIST Handwritten Digit Dataset

MNIST contains grayscale images of handwritten digits from 0 to 9. Each image has spatial dimensions

$$28 \times 28 \times 1.$$

The dataset is particularly suitable for this experiment because the images are small, the classes have visually meaningful structure, and the same image format can be directly supplied to fully connected, convolutional and variational autoencoders.

The dataset can be loaded directly using Keras/TensorFlow:

https://www.tensorflow.org/api_docs/python/tf/keras/datasets/mnist

Recommended laboratory subset:

Use 10,000 training images and 2,000 test images for the main laboratory execution. The full MNIST dataset may be used if computational resources permit.

The labels are not required as reconstruction targets. The original image is the target:

$$x \rightarrow \text{Encoder} \rightarrow z \rightarrow \text{Decoder} \rightarrow \hat{x}$$

where x is the input image and $\hat{x}$ is its reconstruction.

Execution setup actually used. 10,000 MNIST training images were split into 9,000 for training and 1,000 for validation; the first 2,000 MNIST test images were used for testing. Pixels were scaled to $[0, 1]$ (verified range 0.0–1.0). Seed = 42; TensorFlow 2.20.0 / Keras 3.13.2; Adam (10^{-3}), batch size 128, 20 epochs (30 epochs for the VAE).



Figure 1: Sample training digits.

4. Expected Input and Output

Expected Input

Each input image has shape

$$28 \times 28 \times 1.$$

For a batch of 32 images:

$$X \in \mathbb{R}^{32 \times 28 \times 28 \times 1}.$$

Pixel values should be converted from

$$[0, 255] \rightarrow [0, 1].$$

For a fully connected autoencoder, the image may be flattened:

$$28 \times 28 = 784.$$

Therefore,

$$x \in \mathbb{R}^{784}.$$

For a convolutional autoencoder, retain the spatial structure:

$$x \in \mathbb{R}^{28 \times 28 \times 1}.$$

Expected Output

The primary output is a reconstructed image:

$$\hat{x} \in \mathbb{R}^{28 \times 28 \times 1}.$$

Students must display:

Original image
Reconstructed image
Reconstruction error

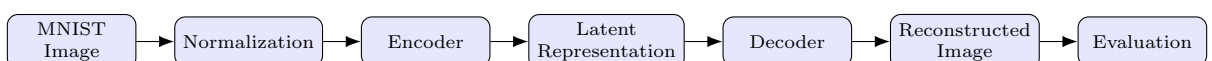
For the denoising model, display:

Original clean image
Noisy image
Denoised reconstruction

For the VAE, display newly generated samples obtained by sampling from the latent distribution.

5. Overall Experimental Pipeline

The complete experiment follows:



The experiment consists of four related studies:

1. Fully Connected Autoencoder
2. Convolutional Autoencoder
3. Denoising Convolutional Autoencoder
4. Variational Autoencoder

6. Exploring Autoencoders

An autoencoder learns to reproduce its input at the output.

The encoder maps the input to a lower-dimensional representation:

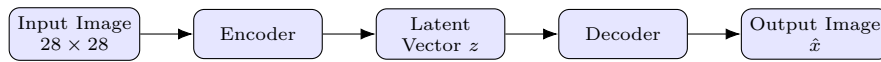
$$z = f_{\theta}(x),$$

and the decoder reconstructs the input:

$$\hat{x} = g_{\phi}(z).$$

The training objective is to minimize reconstruction error:

$$\mathcal{L}_{rec} = \frac{1}{N} \sum_{i=1}^N \|x_i - \hat{x}_i\|^2.$$



The latent vector is a compressed representation of the input.

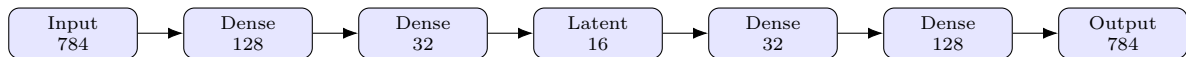
7. Fully Connected Autoencoder

Flatten each image from

$$28 \times 28 = 784$$

to a vector of length 784.

Use the following architecture:



Use ReLU activation in hidden layers and sigmoid activation in the output layer.

Suggested configuration:

Parameter	Value
Latent dimension	16
Optimizer	Adam
Learning rate	10^{-3}
Batch size	128
Epochs	20
Loss	Binary Cross-Entropy

8. Required Analysis of the Basic Autoencoder

Plot 1: Original versus Reconstructed Images

Display at least 10 test images in a grid.

For every image, show:

Original | Reconstruction.

Students must comment on:

- preservation of digit shape;
- loss of fine details;
- blurred regions;
- differences between easy and difficult samples.

Plot 2: Training and Validation Reconstruction Loss

- X-axis: Epoch
- Y-axis: Reconstruction loss
- Curves: Training loss and validation loss

Required inference: Determine whether the model converges and whether there is evidence of overfitting.

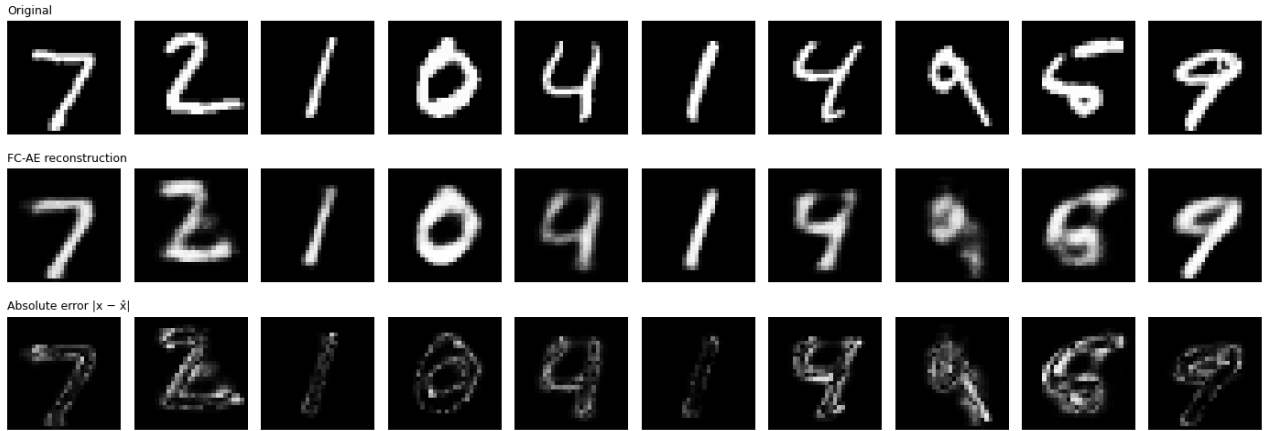


Figure 2: Original (top), FC-AE reconstruction with $d_z = 16$ (middle) and absolute error $|x - \hat{x}|$ (bottom) for ten test images.

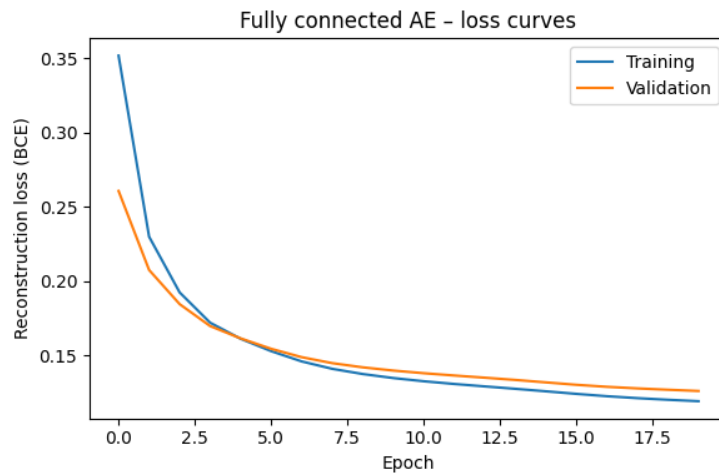


Figure 3: Training and validation reconstruction loss (BCE) of the FC-AE.

Inference - Figure 2 (Original vs. Reconstruction). Shows: ten test digits with their FC-AE reconstructions and the absolute error maps. *Trend:* the digit identity and global shape (slant, loop position, stroke layout) are generally preserved in most samples, e.g. 7, 1, 0 and 4. Thin or intricate strokes are blurred and thickened, fine details such as the tail of the 9 and the hook of the 5 are partly lost, and a faint grey haze appears in the background. The error maps are concentrated along stroke edges. Simple, thick and near-straight digits (1, 0, 7) have good reconstructions, while curvy or thin digits (2, 9, 5, 4) are visibly degraded; the 9 with a long tail becomes a smudge. *Why:* the 16-dimensional dense bottleneck holds only a coarse global description and has no notion of spatial locality, so high-frequency detail is the first thing to be discarded. *Conclusion:* FC-AE captures the coarse, but not fine detail (test MSE 0.0205, SSIM 0.759).

Inference - Figure 3 (Loss curves). Shows: BCE loss per epoch for training and validation data. *Trend:* both curves decrease drastically during the first 5 epochs (training: 0.352 $\rightarrow$ 0.161, validation: 0.261 $\rightarrow$ 0.162), and then the decrease starts to slow down; at epoch 20, the losses are 0.1192 (train) and 0.1261 (validation). The curves are close to each other, intersecting each other around epoch 5, with validation being slightly higher than training from epoch 5 until the end (difference $\approx$ 0.007). *Why:* with only 16 latent units and a small network, the model cannot memorize 9000 training images (the absolute value of BCE is still relatively high, because cross-entropy loss for pixels' probabilities is not binary, therefore it cannot reach zero). *Conclusion:* the model is not overfitting, and it continues to improve until epoch 20; however, since the learning curves decreased slowly after epoch 15, a few more epochs might be enough to improve the performance.

9. Reconstruction Metrics

Students must calculate the following metrics on the test set.

Mean Squared Error

$$MSE = \frac{1}{N} \sum_{i=1}^N (x_i - \hat{x}_i)^2.$$

Mean Absolute Error

$$MAE = \frac{1}{N} \sum_{i=1}^N |x_i - \hat{x}_i|.$$

Structural Similarity

Calculate SSIM between the original and reconstructed images.

The implementation may use:

<https://scikit-image.org/docs/stable/api/skimetrics.html>

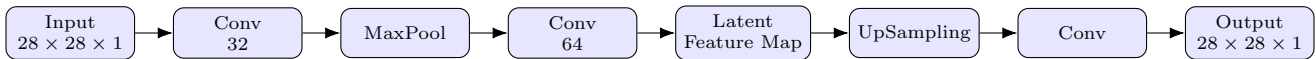
Report:

Metric	Value
Test MSE	0.02046
Test MAE	0.05510
Mean SSIM	0.7587

10. Convolutional Autoencoder

A Convolutional Autoencoder preserves spatial structure and uses convolutional operations in the encoder and upsampling/transposed convolution operations in the decoder.

The conceptual architecture is:



A possible implementation is:

```
Input(28,28,1)
Conv2D(32, 3, activation='relu', padding='same')
MaxPooling2D(2, padding='same')
Conv2D(64, 3, activation='relu', padding='same')
MaxPooling2D(2, padding='same')
Conv2D(64, 3, activation='relu', padding='same')
UpSampling2D(2)
Conv2D(32, 3, activation='relu', padding='same')
UpSampling2D(2)
Conv2D(1, 3, activation='sigmoid', padding='same')
```

The exact implementation may use Conv2DTranspose instead of UpSampling2D.

11. Comparing Fully Connected and Convolutional Autoencoders

Train both models using the same training and test images.

Model	MSE	MAE	SSIM	Parameters
Fully Connected AE	0.02046	0.05510	0.7587	211,040
Convolutional AE	0.00260	0.01509	0.9735	74,497

Plot 3: Reconstruction Comparison

Display:

Original → FC-AE Reconstruction → CAE Reconstruction.

Required inference:

Discuss how preserving spatial locality through convolution affects image reconstruction.

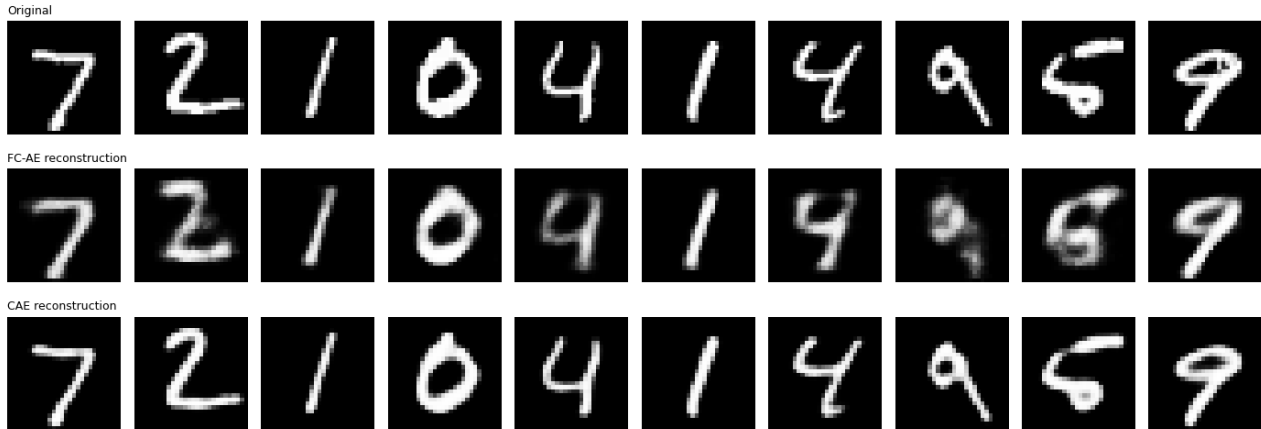


Figure 4: Plot 3: Original (top), FC-AE reconstruction (middle) and CAE reconstruction (bottom).

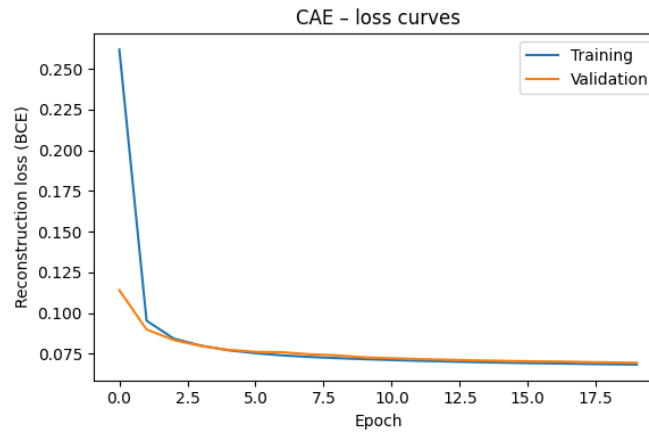


Figure 5: Training/validation BCE loss of the CAE (final: 0.0684 / 0.0695).

Inference – FC vs. Convolutional reconstruction. Shows: the same 10-digit test set reconstructed by both models, plus the metrics table. *Trend:* the CAE reconstructs them all very accurately, including fine details like the narrow strokes and the tail of the 9, whereas the FC-AE reconstructions are often blurry and ambiguous (especially the 9 and 5). Quantitatively, the CAE has $7.9\times$ lower MSE (0.00260 vs 0.02046), $3.7\times$ lower MAE, and a much higher SSIM (0.974 vs 0.759), while utilizing $2.8\times$ fewer parameters (74,497 vs 211,040). *Why:* convolutions utilize small, local receptive fields with shared weights, so the same edge/stroke detectors are used at all positions in the image, preserving the neighboring-pixel structure through pooling/upsampling; the dense architecture processes each pixel independently and must learn all spatial relations separately. One caveat is that the CAE's bottleneck ($7 \times 7 \times 64 = 3136$), actually, is larger than the 784 pixels in the input, and so it is not as compressive as the 16-D latent space of the FC-AE; part of this network's advantage is from the larger bottleneck, not only the convolution operation. *Conclusion:* preserving the spatial locality is a much more effective encoding strategy for images, using far fewer parameters.

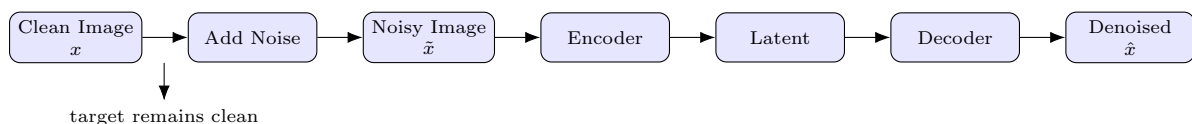
12. Denoising Autoencoder

A denoising autoencoder receives a corrupted image but is trained to reconstruct the original clean image.

The training mapping is:

$$\tilde{x} \rightarrow \text{Encoder} \rightarrow z \rightarrow \text{Decoder} \rightarrow \hat{x}$$

where $\tilde{x}$ is a noisy version of the clean image x .



13. Adding Image Noise

Use at least two corruption types.

Gaussian Noise

$$\tilde{x} = x + n, \quad n \sim \mathcal{N}(0, \sigma^2).$$

Suggested standard deviations:

$$\sigma \in \{0.1, 0.2, 0.3\}.$$

Clip the resulting image to the range $[0, 1]$.

Salt-and-Pepper Noise

Randomly replace a small proportion of pixels with values close to 0 or 1.

Suggested corruption levels:

$$p \in \{0.05, 0.10, 0.20\}.$$

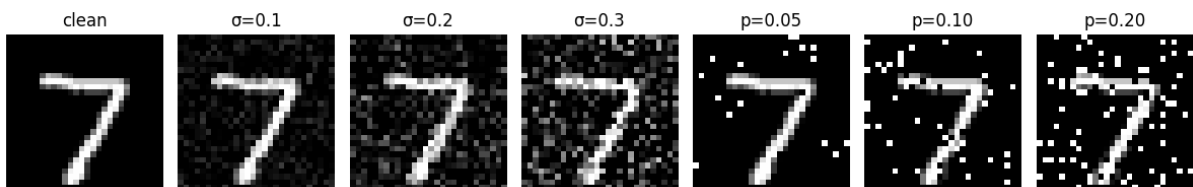


Figure 6: One digit corrupted with Gaussian noise ($\sigma = 0.1, 0.2, 0.3$) and salt-and-pepper noise ($p = 0.05, 0.10, 0.20$).

Denoising training protocol used. Two denoising CAEs were trained with the same architecture as the CAE: one trained on Gaussian noise ($\sigma = 0.2$) and one trained on salt-and-pepper noise ($p = 0.10$); fresh noise was drawn every epoch and the target was always the clean image. The models were then evaluated on all three noise levels of both noise types (against the clean test image).

14. Required Denoising Plots

Plot 4: Clean vs. Noisy vs. Denoised

For at least 10 images, display:

Clean | Noisy | Denoised.

Plot 5: Noise Level vs. Reconstruction Quality

For each noise level, calculate:

$$MSE, \quad MAE, \quad SSIM.$$

Report:

Noise Level	MSE	MAE	SSIM
0.1	0.0037	0.0180	0.9588
0.2	0.0047	0.0215	0.9443
0.3	0.0072	0.0294	0.8747

Required inference:

Explain how increasing corruption affects reconstruction quality and identify whether the denoising model is able to recover the major structure of the original image.

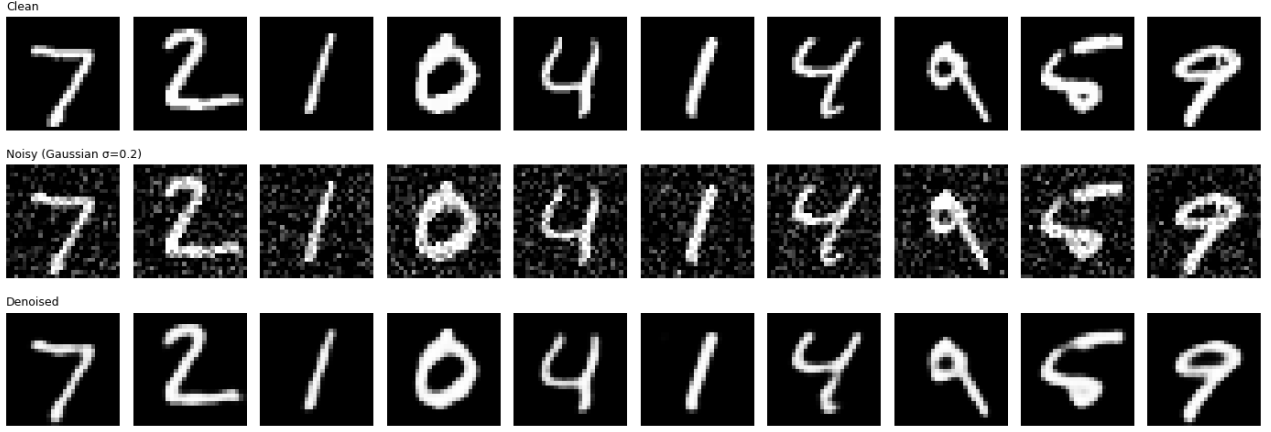


Figure 7: Plot 4a: Clean / noisy (Gaussian $\sigma = 0.2$) / denoised.

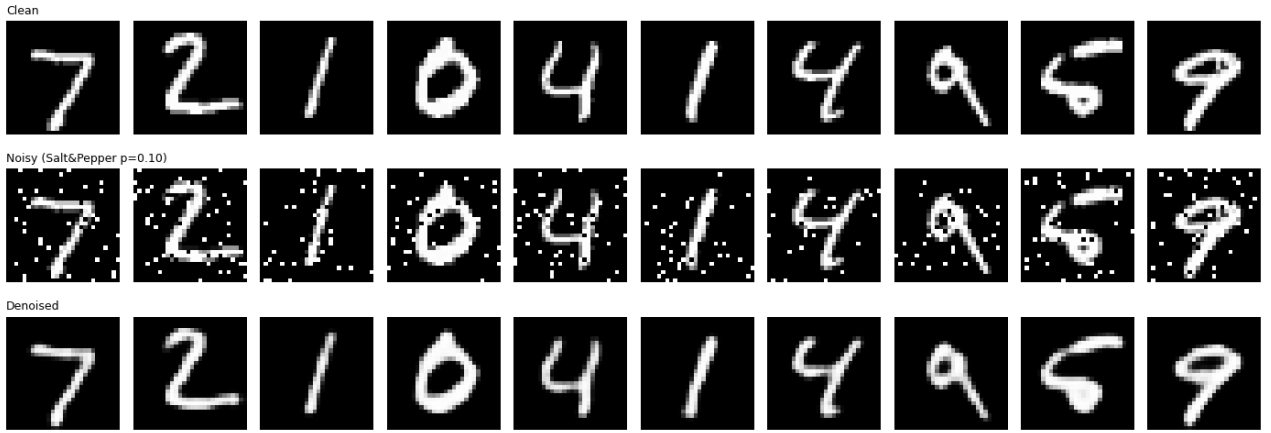


Figure 8: Plot 4b: Clean / noisy (salt-and-pepper $p = 0.10$) / denoised.

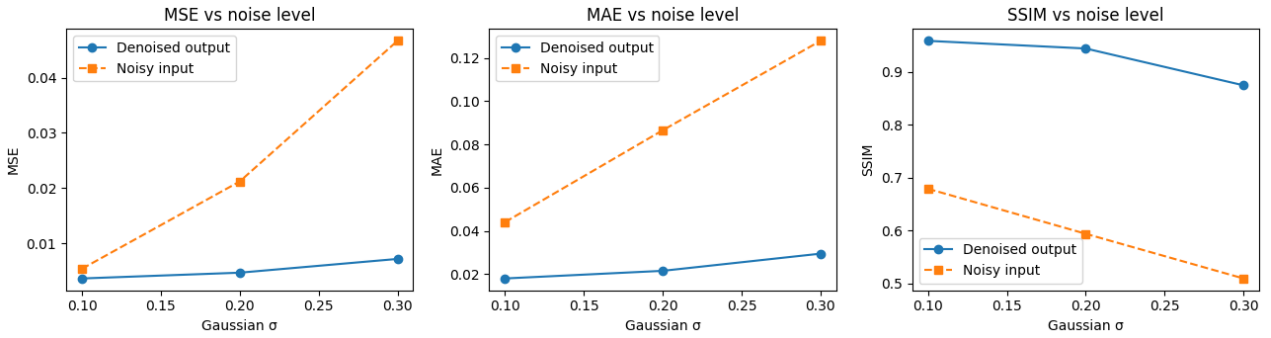


Figure 9: Plot 5a: MSE, MAE and SSIM versus Gaussian noise level (denoiser trained at $\sigma = 0.2$). Each metric uses its own axis.

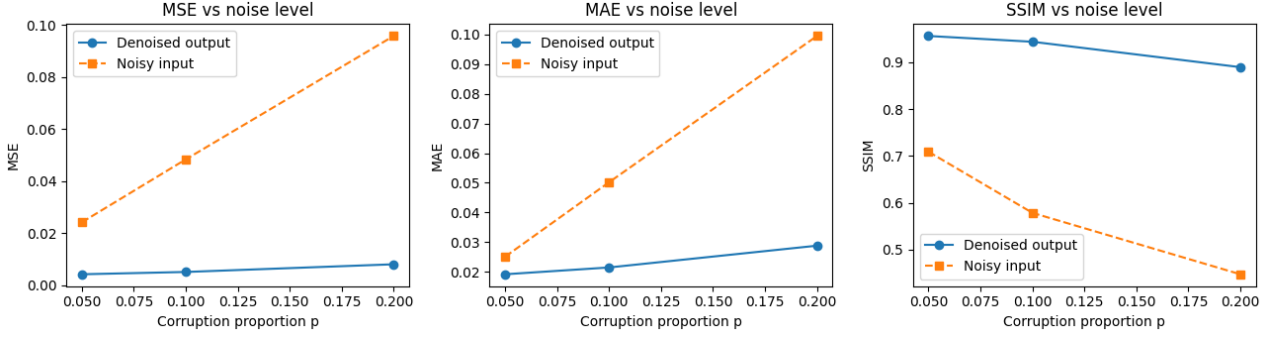


Figure 10: Plot 5b: MSE, MAE and SSIM versus salt-and-pepper level (denoiser trained at $p = 0.10$).

Detailed results (test set, 2,000 images; all metrics against the clean image).

Denoiser trained on	Test noise	Noisy MSE	Noisy SSIM	Den. MSE	Den. MAE	Den. SSIM
Gaussian $\sigma = 0.2$	Gaussian 0.1	0.0054	0.679	0.0037	0.0180	0.959
	Gaussian 0.2	0.0212	0.594	0.0047	0.0215	0.944
	Gaussian 0.3	0.0466	0.509	0.0072	0.0294	0.875
	S&P 0.05	0.0241	0.709	0.0049	0.0214	0.933
	S&P 0.10	0.0483	0.579	0.0071	0.0273	0.884
	S&P 0.20	0.0958	0.448	0.0134	0.0422	0.760
S&P $p = 0.10$	S&P 0.05	0.0241	0.709	0.0041	0.0191	0.956
	S&P 0.10	0.0483	0.579	0.0050	0.0214	0.944
	S&P 0.20	0.0958	0.448	0.0080	0.0288	0.890
	Gaussian 0.1	0.0054	0.679	0.0073	0.0300	0.874
	Gaussian 0.2	0.0212	0.594	0.0130	0.0490	0.694
	Gaussian 0.3	0.0466	0.509	0.0203	0.0707	0.600

Inference – Noise level vs. reconstruction quality and structure recovery. *Shows:* reconstruction error and similarity against the clean image for increasing corruption, for the noisy input itself and for the denoised output. *Trend:* for Gaussian noise the denoised MSE rises monotonically ($0.0037 \rightarrow 0.0047 \rightarrow 0.0072$), MAE rises ($0.0180 \rightarrow 0.0294$) and SSIM falls ($0.959 \rightarrow 0.944 \rightarrow 0.875$) as σ increases from 0.1 to 0.3. The degradation of the denoised output is much slower than that of the noisy input, whose MSE increases about $9\times$ ($0.0054 \rightarrow 0.0466$) and SSIM falls from 0.68 to 0.51. The denoiser removes about 31%, 78% and 85% of the input MSE at $\sigma = 0.1, 0.2, 0.3$, and raises SSIM by 0.28–0.37. The same trend holds for salt-and-pepper noise (SSIM $0.956 \rightarrow 0.890$ for p from 0.05 to 0.20). At the lowest level ($\sigma = 0.1$) the gain is smallest because the input is already almost clean and the model’s own reconstruction error (clean-input CAE MSE 0.0026) is a floor. *Why:* the network learns the manifold of digit-like images; noise lies off this manifold and is discarded by the bottleneck and the pooling, but as noise grows, strokes become ambiguous, so some are reconstructed incorrectly or thinned. $\sigma = 0.3$ also lies outside the training noise level ($\sigma = 0.2$). The models are noise-specific: the Gaussian-trained model reaches SSIM 0.760 on S&P $p = 0.20$, and the S&P-trained model only 0.600 on Gaussian $\sigma = 0.3$, i.e. much worse than the matched denoisers (0.890 and 0.875). *Conclusion:* the denoising CAE recovers the major digit structure well up to moderate noise (SSIM > 0.94 at $\sigma \leq 0.2$), loses fine detail at high noise, and generalises best to the noise type it was trained on.

15. Variational Autoencoder

A conventional autoencoder maps an input to a deterministic latent vector:

$$z = f(x).$$

A VAE instead learns a probability distribution:

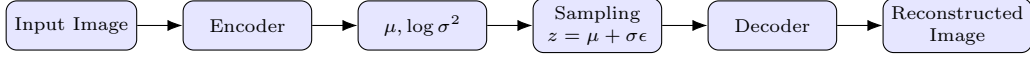
$$q_\phi(z|x) = \mathcal{N}(\mu(x), \text{diag}(\sigma^2(x))).$$

The encoder produces:

$$\mu, \quad \log \sigma^2.$$

A latent vector is sampled using the reparameterization trick:

$$z = \mu + \sigma \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$$



16. VAE Loss Function

The VAE objective consists of two components.

Reconstruction Loss

$$\mathcal{L}_{rec} = -\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)].$$

For normalized MNIST images, binary cross-entropy can be used.

KL Divergence

$$D_{KL}(q_\phi(z|x) \parallel p(z)) = -\frac{1}{2} \sum_j (1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2).$$

The total loss is

$$\mathcal{L}_{VAE} = \mathcal{L}_{rec} + \mathcal{L}_{KL}$$

where the prior is usually

$$p(z) = \mathcal{N}(0, I).$$

17. VAE Implementation

Use a small latent space so that it can be visualized.

Suggested latent dimension:

$$z \in \mathbb{R}^2.$$

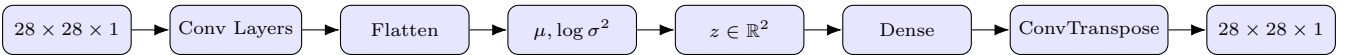
The encoder should produce:

$$\mu \in \mathbb{R}^2, \quad \log \sigma^2 \in \mathbb{R}^2.$$

The decoder maps

$$z \rightarrow 28 \times 28 \times 1.$$

A compact architecture is:



Implementation used. Encoder: Conv(32, stride 2) → Conv(64, stride 2) → Flatten → Dense(16) → $(\mu, \log \sigma^2)$ in $\mathbb{R}^2$ (69,076 parameters). Decoder: Dense(7·7·64) → Reshape → ConvTranspose(64) → ConvTranspose(32) → ConvTranspose(1, sigmoid) (65,089 parameters). Total 134,165 parameters. Loss = per-image summed BCE + KL ($\beta = 1$), Adam 10^{-3} , batch 128, 30 epochs. Reconstruction metrics are computed from the decoded posterior mean μ .

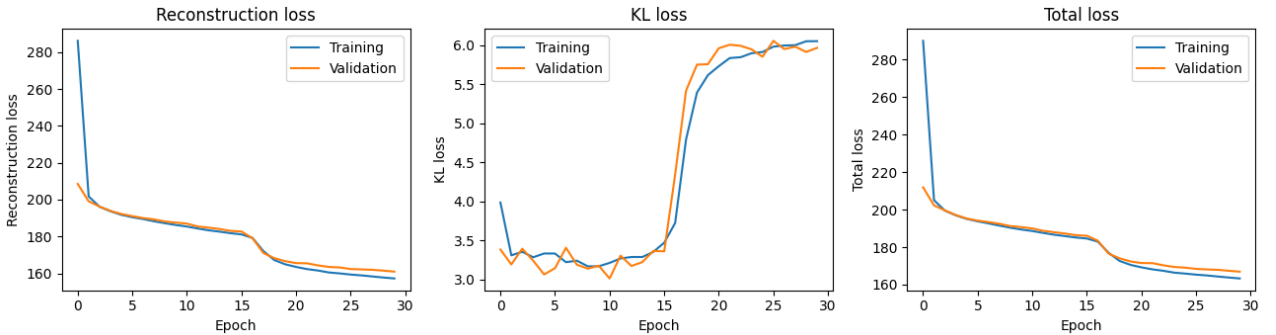


Figure 11: VAE training and validation reconstruction, KL and total loss (Plot 9 in the required list).

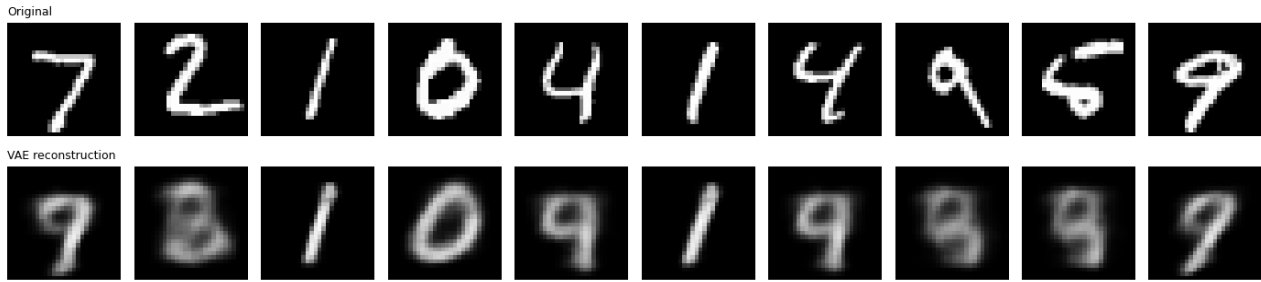


Figure 12: VAE reconstructions (decoded from posterior means) of the first ten test images.

Inference – VAE training and reconstruction. *Shows:* the reconstruction loss, the KL loss and their sum on training and validation data, and the VAE reconstructions. *Trend:* the reconstruction loss falls from 286.1 to 157.2 (train) and from 208.5 to 160.9 (validation); the KL term stays at about 3.2–3.4 until epoch 16, then increases to about 6.0 as the reconstruction loss drops sharply (epochs 17–20), and then both flatten. Validation and training curves stay close (final validation total 166.9 vs training 163.3), so there is no overfitting. On the test set the reconstruction loss is 156.15, the KL loss 6.10 and the total 162.26. Reconstructions are blurry and sometimes change the digit class (7, 4 and 5 are rendered as 9-like shapes; 2 becomes an 8-like blob); test SSIM is only 0.440 and MSE 0.0464. *Why:* the KL term pulls each posterior toward $\mathcal{N}(0, I)$ and penalises information stored in z ; together with the very small 2-D bottleneck, the VAE can only keep a coarse class/style code, and the BCE-trained decoder outputs blurry averages for uncertain codes. Notably, the 2-D deterministic FC-AE of Section 27 reaches a similar MSE (0.0459) and SSIM (0.455), so the poor VAE scores are mainly due to the 2-D bottleneck rather than the KL regulariser. *Conclusion:* the VAE trades reconstruction fidelity for a structured, samplable latent space; it is not comparable in fidelity to the CAE (MSE 0.0026), which has a latent of 3136 values.

18. Exploring the VAE Latent Space

Since the latent dimension is two, students can visualize the encoded test images in a two-dimensional plane.

Plot 6: VAE Latent Space

- X-axis: z_1
- Y-axis: z_2
- Color/marker category: digit label

The labels are used only for visualization; they are not supplied to the VAE during training.

Required inference:

Students should describe whether samples belonging to the same digit occupy similar regions and whether different digits overlap.

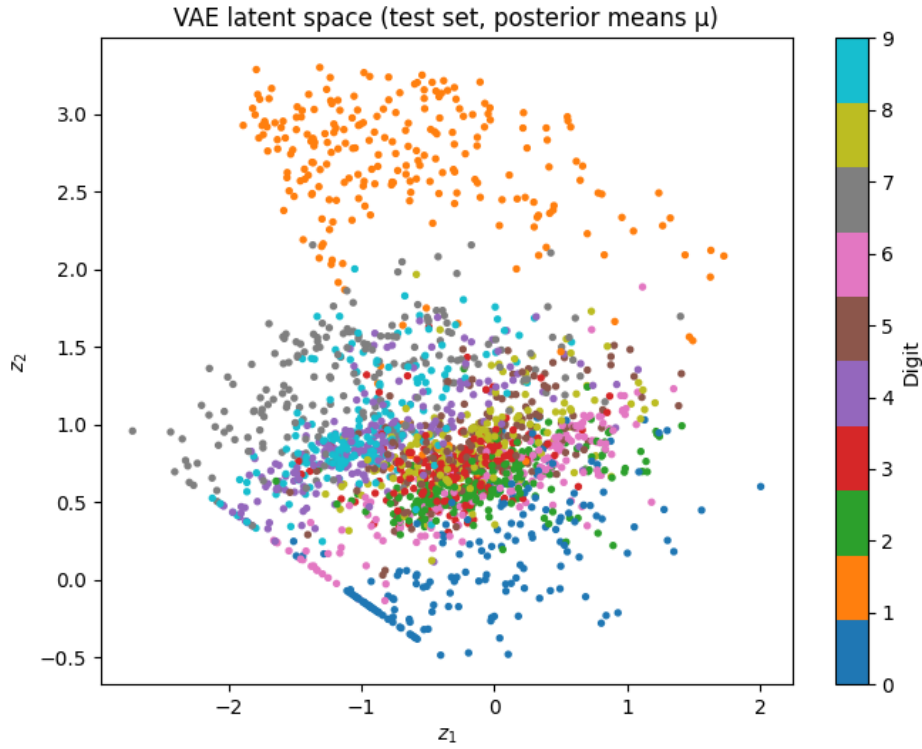


Figure 13: Plot 6: Two-dimensional VAE latent space (test set, posterior means μ), coloured by digit label.

Inference – VAE latent space. *Shows:* the 2-D posterior means of the 2,000 test images coloured by label (labels were not used in training). *Trend:* the latent distribution is compact and roughly centred around the prior: mean of $\mu = (-0.44, 1.00)$, standard deviation of $\mu = (0.73, 0.72)$, and the average posterior σ is very small $(0.089, 0.040)$, so each image is encoded as a confident, tight cluster. Digits 0 (bottom) and 1 (top) form distinct, well-separated regions, with 7 (upper left) and 6 (lower left/right edge) partly separated. In contrast, 2, 3, 5 and 8 overlap heavily near the centre, and 4, 7 and 9 overlap in the upper-left/centre region. *Why:* with only two latent dimensions, 10 classes cannot be separated linearly; digits with simple, very different shapes (0, 1) separate, while visually similar digits share one region. The KL term compresses everything towards the origin. *Conclusion:* the latent space is organised by visual similarity rather than by class label (unsupervised), with the same digits occupying similar but overlapping regions.

19. Generating New Images with the VAE

Sample points from

$$z \sim \mathcal{N}(0, I).$$

Pass the sampled latent vectors through the decoder.

Plot 7: Randomly Generated Images

Generate at least 25 images using randomly sampled latent vectors.

Display them in a grid.

Students should determine:

- whether the generated images resemble handwritten digits;
- whether digit shapes are recognizable;
- whether there is visible variation among generated samples; and
- whether some samples are ambiguous or unrealistic.

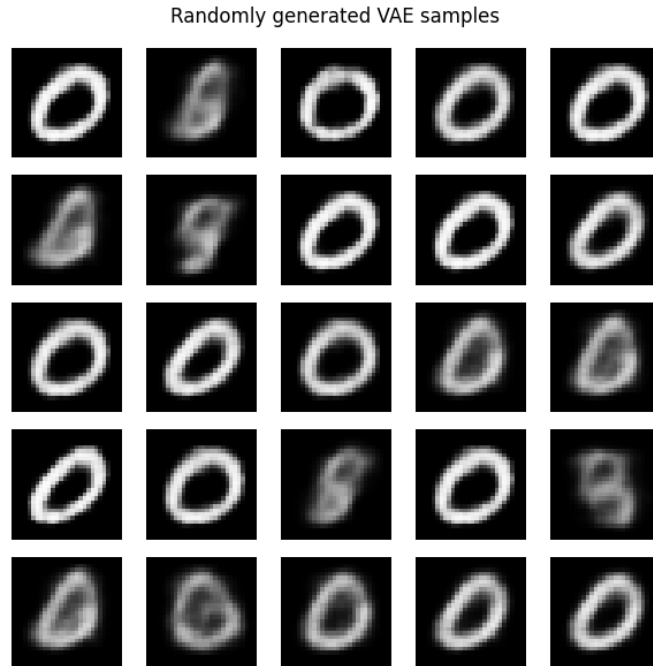


Figure 14: Plot 7: 25 images generated by decoding $z \sim \mathcal{N}(0, I)$.

20. Latent Space Interpolation

Select two latent vectors:

$$z_A, \quad z_B.$$

Construct

$$z(\alpha) = (1 - \alpha)z_A + \alpha z_B, \quad \alpha \in [0, 1].$$

Use values such as:

$$\alpha = 0, 0.1, 0.2, \dots, 1.$$

Decode every interpolated vector.

Plot 8: Latent Space Interpolation

Display the decoded images in sequence.

Required inference:

Describe whether the decoded image changes smoothly as the latent vector moves between two points.

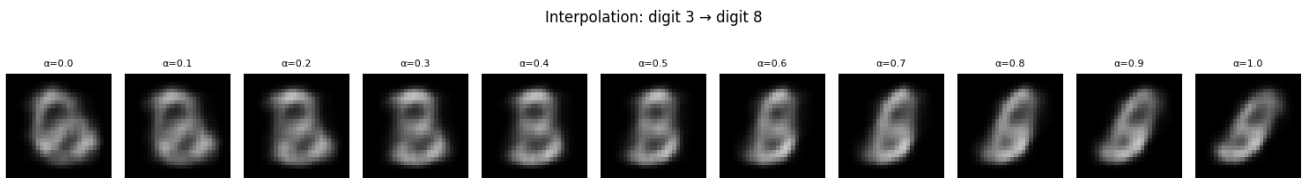


Figure 15: Plot 8: Decoded images for $z(\alpha) = (1 - \alpha)z_A + \alpha z_B$, $\alpha = 0, 0.1, \dots, 1$ (from a test “3” to a test “8”).

Inference – Latent interpolation. *Shows:* eleven decoded images along the straight line between the latent means of the first test “3” and the first test “8”. *Trend:* the image changes gradually and continuously: a rounded, 8-like blob at $\alpha = 0$ gradually loses its upper loop and becomes a slanted, 5/S-like shape at $\alpha = 1$; there are no abrupt jumps or noisy intermediate frames. Both endpoints are blurry, and neither looks exactly like a clean 3 or 8. *Why:* the KL regulariser and the noise injected by the reparameterisation make the decoder map nearby latent points to similar images, so the latent space is continuous. Digits 3 and 8 occupy the same overlapping central region in Plot 6, hence the decoded endpoints are blurry averages. *Conclusion:* the interpolation is smooth, which shows that the VAE has learned a continuous latent manifold rather than memorising isolated points.

21. Reconstruction Comparison Across Models

Compare the three reconstruction-based models:

Model	MSE	MAE	SSIM	Parameters
Fully Connected AE	0.02046	0.05510	0.7587	211,040
Convolutional AE	0.00260	0.01509	0.9735	74,497
Denoising CAE ($\sigma = 0.2$)	0.00473	0.02150	0.9443	74,497

For the VAE, report its reconstruction loss separately because its total objective also contains the KL-divergence term.

VAE Metric	Value
Reconstruction Loss (summed BCE per image)	156.154
KL Loss	6.103
Total Loss	162.257
Test MSE	0.04645
Test MAE	0.10986
Mean SSIM	0.4402

22. Required Plots

The final report must contain the following plots:

1. Original image versus reconstructed image.
2. Training and validation loss for the basic autoencoder.
3. Fully connected AE versus convolutional AE reconstruction.
4. Clean versus noisy versus denoised images.
5. Noise level versus MSE/MAE/SSIM.
6. VAE latent-space visualization.
7. Randomly generated VAE images.
8. Latent-space interpolation.
9. Training/validation reconstruction loss for the VAE.
10. Reconstruction-error distribution on the test set.

For plots containing multiple quantitative measures, use separate figures or clearly labeled axes rather than placing incompatible scales on one axis.

23. Reconstruction Error Distribution

Calculate the per-image reconstruction error:

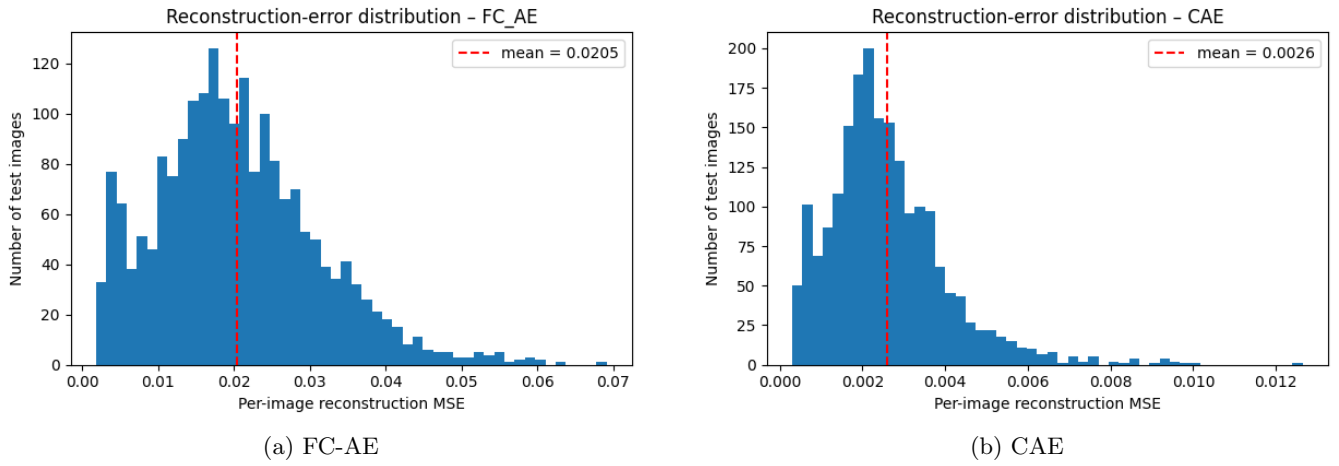
$$e_i = \frac{1}{D} \sum_{j=1}^D (x_{ij} - \hat{x}_{ij})^2.$$

Plot 9: Distribution of Reconstruction Errors

Plot a histogram of e_i over the test set.

Students should identify:

- the typical reconstruction-error range;
- whether the distribution is concentrated or spread out;
- possible high-error samples; and
- representative images corresponding to high reconstruction error.



Plot 9: Distribution of per-image reconstruction error e_i on the 2,000 test images.

Model	Mean	Median	Std	5–95% range	Max
FC-AE	0.02046	0.01932	0.01066	0.00435–0.03936	0.06911
CAE	0.00260	0.00236	0.00143	0.00066–0.00523	0.01264

Inference – Error distribution. Shows: histograms of per-image MSE. Trend: overall distributions are unimodal, slightly right skewed with a single peak close to the mean; typical errors are in 0.004–0.039 range for the FC-AE and 0.0007–0.0052 for the CAE (the CAE range is about $8\times$ lower). Spread is moderate (std/mean ≈ 0.5) and the long tail contains a few high-error images (max 0.069 for FC-AE, 0.0126 for CAE). Worst CAE image (0.0126) is better than the median FC-AE image (0.0193). Why: most digits are ordinary and reconstructed at a similar level; tail comes from unusual shapes (see Section 24). Conclusion: average error represents typical behaviour well; error and visual quality are clearly related, because the high-error images are those that look blurred or wrong (Section 24), while low-error images are practically indistinguishable from the originals (Plot 3).

24. Required High-Error Image Analysis

Select the five test images having the largest reconstruction errors.

Display:

Sample	Fully Connected AE			Convolutional AE		
	Test idx	Digit	Error e_i	Test idx	Digit	Error e_i
1	1782	8	0.06911	895	0	0.01264
2	1758	8	0.06298	1982	6	0.01005
3	1170	8	0.06103	1526	0	0.00993
4	744	2	0.06076	1594	9	0.00957
5	338	8	0.05965	431	8	0.00952

The original and reconstructed images for these samples are shown below (top row: original, bottom row: reconstruction; left to right = Sample 1 to 5).

Students should provide a short explanation for why these images may be difficult to reconstruct.



Figure 16: Five highest-error test images, FC-AE (top: original, bottom: reconstruction).



Figure 17: Five highest-error test images, CAE (top: original, bottom: reconstruction).

Inference – High-error images. Four of the five worst images are the digit 8 (one is a looping two), which are written in an unusual, looping or crossing manner, have very thin strokes with several closed loops or are tilted, such that a 16-D code can't encode this information exactly. The FC-AE reconstructions are blurry and in case of some samples appear to look like a different digit (e.g. sample 1 and 3 becomes 7/2-like). For the CAE, the hardest images to reconstruct are two 0s with thick walls or an internal stroke, a 6 with an extra stroke, a 9 with an unusual tail and an 8 with a thin, wide shape. Their reconstructions are still clearly discernible but smoothed and the extra internal stroke or the sharp thin curves partially gone. Rare images in the training set (written in an atypical style, have extra strokes, very thick or very thin lines) appear difficult because the model has learned a compact description for its most common shapes.

25. Consolidated Results

Model	MSE	MAE	SSIM	Parameters	Time
FC Autoencoder	0.02046	0.05510	0.7587	211,040	15.9 s
Conv. Autoencoder	0.00260	0.01509	0.9735	74,497	19.0 s
Denoising CAE	0.00473	0.02150	0.9443	74,497	23.3 s
VAE ($d_z = 2$)	0.04645	0.10986	0.4402	134,165	27.3 s

26. Required Inferences

For every major plot, students must write a short inference containing:

1. What does the plot show?
2. What trend is observed?
3. Why might the observed trend occur?
4. What conclusion can be drawn?

The following inferences are mandatory:

- effect of latent dimension on reconstruction;
- difference between fully connected and convolutional reconstruction;
- effect of Gaussian noise level;
- ability of the denoising autoencoder to recover image structure;
- characteristics of the VAE latent distribution;
- structure of the two-dimensional VAE latent space;
- quality and diversity of generated samples;
- smoothness of latent-space interpolation;
- relationship between reconstruction error and visual quality; and
- differences between deterministic and probabilistic latent representations.

Mandatory inferences (summary of the experiment)

1. **Effect of latent dimension on reconstruction.** Test MSE falls from 0.0459 ($d_z = 2$) to 0.0202 ($d_z = 32$) and SSIM rises from 0.455 to 0.761, with diminishing returns (42%, 19%, 6% MSE reductions per doubling) because a larger code stores more detail but the model is then limited by its decoder and by what is learnable. Conclusion: a larger latent is better but with decreasing benefit, and $d_z = 2$ is far too small for faithful reconstruction.
2. **Fully connected vs. convolutional reconstruction.** The CAE has $7.9\times$ lower MSE (0.0026 vs 0.0205) and SSIM 0.974 vs 0.759 with $2.8\times$ fewer parameters, because local weight-shared filters preserve spatial structure while dense layers ignore pixel adjacency. Its larger latent map (3136 values) also contributes.

3. **Effect of Gaussian noise level.** As σ increases from 0.1 to 0.3, the denoised MSE grows $0.0037 \rightarrow 0.0072$, MAE $0.0180 \rightarrow 0.0294$ and SSIM drops $0.959 \rightarrow 0.875$, since heavier noise destroys more of the stroke information; degradation is gradual, not catastrophic.
4. **Ability of the DAE to recover image structure.** Plot 4 shows that the digit shape and background are recovered from visibly noisy inputs; for $\sigma = 0.2$ the SSIM improves from 0.594 (noisy input) to 0.944 and the MSE drops from 0.0212 to 0.0047, because the autoencoder learns the digit manifold and treats noise as non-structure. Fine strokes are slightly smoothed.
5. **Characteristics of the VAE latent distribution.** The posterior means are centred near the prior (mean $\approx (-0.44, 1.0)$, std ≈ 0.72 per axis), and the posterior standard deviations are small (0.09, 0.04); the KL is low (6.1 nats). The regulariser keeps the code compact and close to $\mathcal{N}(0, I)$, but the aggregate posterior is not an exact standard normal.
6. **Structure of the 2-D VAE latent space.** 0 and 1 occupy separate regions, 7, 4 and 9 form a partly overlapping group, and 2, 3, 5, 8 overlap heavily in the centre, so the structure follows visual similarity; the two-dimensional bottleneck is too small to separate all ten digits.
7. **Quality and diversity of generated samples.** Generated samples are recognisable digit-like images, but diversity is low: roughly two-thirds are the digit 0, and the rest are blurry ambiguous blobs. This is due to the large area of the latent space that decodes to a 0 and to blurry averaging at class boundaries.
8. **Smoothness of latent interpolation.** Decoded images change continuously from an 8-like to a 5/S-like shape between $\alpha = 0$ and $\alpha = 1$ without sudden jumps, showing that the VAE's latent space is continuous because of the KL regularisation and the sampling noise.
9. **Relationship between reconstruction error and visual quality.** Lower error corresponds to visually sharper reconstructions: CAE (MSE 0.0026, SSIM 0.97) is nearly indistinguishable from the originals, FC-AE (0.0205, 0.76) is blurred and the VAE (0.0464, 0.44) is blurriest and sometimes changes the digit. High-error images within a model are exactly the ones that appear most distorted. MSE alone is imperfect, so SSIM was reported too.
10. **Deterministic vs. probabilistic latent representations.** The AE maps each image to one point, giving the best reconstruction at a given size but no guarantee that points between codes decode to valid digits; the VAE maps each image to a distribution regularised towards $\mathcal{N}(0, I)$, which gives a continuous, samplable latent space (random generation and smooth interpolation) at the cost of blurrier reconstruction (loss of fidelity and KL cost).

27. Additional Latent-Dimension Study

Repeat the basic autoencoder using:

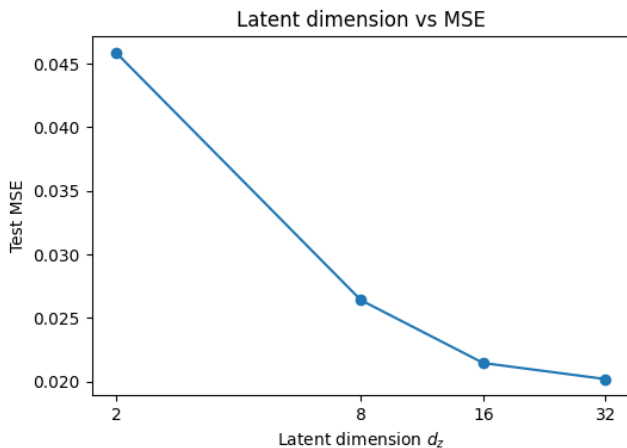
$$d_z \in \{2, 8, 16, 32\}.$$

Report:

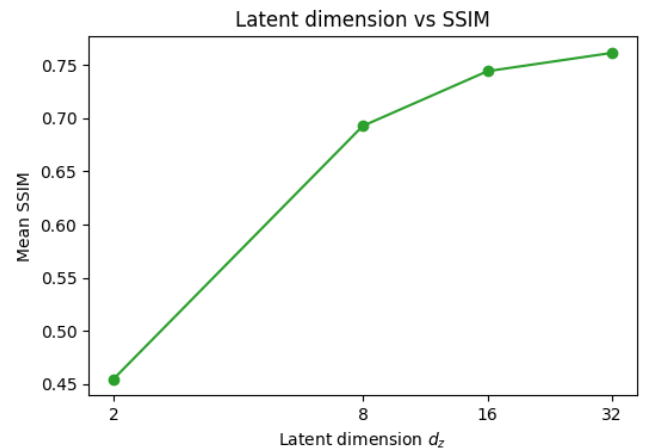
Latent Dimension	MSE	SSIM	Parameters
2	0.04586	0.4548	210,130
8	0.02640	0.6927	210,520
16	0.02145	0.7441	211,040
32	0.02018	0.7613	212,080

Plot 10: Latent Dimension vs. Reconstruction MSE

Students should explain the trade-off between compression and reconstruction quality.



(a) Latent dimension vs. test MSE (Plot 10)



(b) Latent dimension vs. SSIM

Inference – Latent-dimension trade-off. Shows: test MSE and SSIM of the FC-AE for $d_z \in \{2, 8, 16, 32\}$.
Trend: MSE decreases monotonically ($0.0459 \rightarrow 0.0264 \rightarrow 0.0215 \rightarrow 0.0202$) and SSIM increases ($0.455 \rightarrow 0.693$)

$\rightarrow 0.744 \rightarrow 0.761$). Returns diminish: $2 \rightarrow 8$ reduces MSE by 42%, $8 \rightarrow 16$ by 19% and $16 \rightarrow 32$ by only 6%. The parameter count hardly changes ($210,130 \rightarrow 212,080$) because it is dominated by the 784×128 and 128×784 layers. (The $d_z = 16$ result, 0.02145, differs slightly from the main run, 0.02046, because it is a separate training run with different random initialisation.) *Why:* a very small bottleneck cannot store enough information about stroke thickness, slant and shape, so images collapse to blurry class prototypes; beyond ≈ 16 dimensions the remaining error is mainly high-frequency detail that a dense decoder cannot recover anyway. *Conclusion:* a smaller latent gives stronger compression but poorer reconstruction; a moderate $d_z \approx 16$ captures most of the achievable quality.

28. Discussion Questions

1. What is an autoencoder?

Answer: A neural network that has been trained to reconstruct its inputs at the output by passing them through a narrow bottleneck, forcing it to learn a compact representation: $x \rightarrow z \rightarrow \hat{x}$.

2. What is the purpose of the encoder?

Answer: To encode the input image into a lower-dimensional latent representation $z = f_\theta(x)$ containing the same key information.

3. What is the purpose of the decoder?

Answer: To decode the latent code into a reconstructed image, $\hat{x} = g_\phi(z)$ (in a VAE, to also generate images from codes sampled at test time).

4. What is a latent representation?

Answer: The compressed latent feature vector (or feature map) produced by the encoder; in this lab, 16 numbers in the case of the FC-AE and 2 numbers for the VAE.

5. Why is the latent representation usually smaller than the input?

Answer: The bottleneck forces the network to learn to encode the useful structure in the input rather than just copying it over (i.e. allows for compression, noise reduction, feature learning), without the narrow layer the network would just learn an identity function.

6. What happens when the latent dimension is too small?

Answer: Information gets lost during encoding resulting in blurry class-averaged reconstructions. At $d_z = 2$ the MSE is 0.0459 and SSIM 0.455 (compared to 0.0202 and 0.761 at $d_z = 32$).

7. What happens when the latent dimension is increased?

Answer: Improves reconstruction (lower MSE, higher SSIM) with diminishing returns, but less compression; too high and the network can learn to encode-decode an essentially identity function, and hence learn less interesting features.

8. Why can a convolutional autoencoder preserve spatial information more effectively?

Answer: The local receptive fields and shared weights allow to exploit neighbour pixel correlations and the translation structure in the image (in contrast to the FC-AE's fully connected layers), and the latent encodes a spatial feature map; here the SSIM is 0.974 for the CAE compared to 0.759 for the FC-AE with $2.8\times$ times fewer parameters.

9. Differentiate an autoencoder and a denoising autoencoder.

Answer: A normal AE recovers its own (clean) input; a DAE is trained on a corrupted input $\tilde{x}$ to output the clean x , learning meaningful features and having to remove the noise.

10. Why is the clean image used as the target in denoising?

Answer: To learn the mapping noisy $\rightarrow$ clean. With a noisy target, it would learn to replicate the noise; the clean target teaches it the underlying structure.

11. What happens when the noise level is increased?

Answer: The noisy input deteriorates strongly (SSIM $0.68 \rightarrow 0.51$ for σ $0.1 \rightarrow 0.3$), while the denoised output also deteriorates, but more slowly (SSIM $0.959 \rightarrow 0.875$; MSE $0.0037 \rightarrow 0.0072$).

12. Why are MSE, MAE and SSIM useful for image reconstruction?

Answer: MSE penalizes large pixel errors, MAE measures average absolute deviation and is less sensitive to outliers, and SSIM compares local luminance, contrast and structure and is closer to human perception; together they give complementary views.

13. What is a Variational Autoencoder?

Answer: A generative autoencoder whose encoder outputs the parameters $(\mu, \log \sigma^2)$ of a distribution $q_\phi(z|x)$; z is sampled and decoded, and training minimizes reconstruction loss plus KL divergence to a prior.

14. How does a VAE differ from a conventional autoencoder?

Answer: A conventional AE has a deterministic code $z = f(x)$ and only a reconstruction loss; a VAE has a stochastic code from a learned distribution, a KL regulariser, and a latent space that can be sampled to generate new images.

15. Why does a VAE learn a distribution instead of a single latent vector?

Answer: To make nearby latent points decode to similar images and to make the whole region near the prior meaningful, so that the decoder can be used generatively; a single point per image would leave gaps.

16. What is the reparameterization trick?

Answer: Writing $z = \mu + \sigma \odot \epsilon$ with $\epsilon \sim \mathcal{N}(0, I)$, which moves the randomness to an external input so that gradients can flow through μ and σ by backpropagation.

17. Why is ϵ sampled from a standard normal distribution?

Answer: Because it is a fixed, parameter-free noise source with zero mean and unit variance; scaling and shifting it gives samples from $\mathcal{N}(\mu, \sigma^2)$ exactly, and it is consistent with the $\mathcal{N}(0, I)$ prior.

18. What is the role of KL divergence in the VAE loss?

Answer: It regularises the posterior $q_\phi(z|x)$ towards the prior $p(z) = \mathcal{N}(0, I)$, which keeps the latent space compact,

continuous and samplable, at the cost of some reconstruction quality. Here it was 6.10 nats against a reconstruction term of 156.15.

19. **What is the prior distribution used in the VAE?**

Answer: The standard normal $p(z) = \mathcal{N}(0, I)$.

20. **Why is a two-dimensional latent space useful for visualization?**

Answer: Because it can be plotted directly (as a scatter plot, Plot 6) and decoded on a grid (manifold), without requiring dimensionality reduction; the cost is a low reconstruction quality.

21. **What does latent-space interpolation demonstrate?**

Answer: That the latent space is continuous and meaningful: moving linearly between two codes results in a gradual morph between the two decoded images (here from 8-like to 5/S-like, with no abrupt jumps).

22. **Why can a VAE generate new images?**

Answer: Because the latent space is regularised towards a known prior, so that sampling $z \sim \mathcal{N}(0, I)$ and decoding yields images in regions the decoder has been trained on.

23. **Why might some VAE-generated images be unclear?**

Answer: That samples fall in between class regions where the decoder averages several digits; the 2-D latent and the BCE/KL trade-off result in blurry outputs, and the aggregate posterior does not exactly match the prior. In our case, many samples were blurry blobs.

24. **What is the difference between reconstruction and generation?**

Answer: Reconstruction encodes a real image and decodes it back ($x \rightarrow z \rightarrow \hat{x}$) and can be compared to the original; generation decodes a latent sampled from the prior, with no ground truth, and creates a new image.

25. **Why should test images remain unseen during training?**

Answer: To give an unbiased estimate of generalisation; if the model saw the test images, the metrics would measure memorisation. Here the 2,000 test images were not used for training or validation (1,000 images were held out for validation).

29. Additional Exercises

1. Train the convolutional autoencoder with latent dimensions of 4, 8, 16 and 32 and compare reconstruction quality.
2. Compare Gaussian noise and salt-and-pepper noise using the same denoising architecture.
3. Train the denoising autoencoder using two different noise levels and compare SSIM.
4. Replace the convolutional decoder with transposed convolutions and compare the reconstructions.
5. Implement a VAE with latent dimension 2 and 8 and compare the latent representations.
6. Investigate the effect of increasing the KL-loss contribution.
7. Generate a larger grid of VAE samples and discuss diversity.
8. Select two digit regions in the latent space and perform interpolation.

Status of the exercises in this run. Exercise 2 (Gaussian vs. salt-and-pepper noise) was performed using the same denoising CAE architecture as specified in the problem description: the matched denoiser achieves an SSIM of 0.944 at $\sigma = 0.2$ / $p = 0.10$, and the cross-noise results confirm that each network benefits most from being trained on the noise type it was exposed to (Plot 5, table in Section 14). Exercise 7 (larger sample grid) is covered by the results for the 25-sample grid and the decoded manifold at 15×15 , and Exercise 8 (interpolation between two digit regions) by the 3→8 interpolation in Section 20.

30. Expected Outcome

At the end of the experiment, students should be able to demonstrate the complete image-representation and reconstruction pipeline:

Image → Encoder → Latent Representation → Decoder → Reconstructed Image

They should also demonstrate denoising:

Clean Image → Noise → Denoising Autoencoder → Recovered Image

and variational generation:

Image → (μ, σ) → z → Decoder → Generated/Reconstructed Image

The final report should contain both quantitative evaluation and visual analysis. Numerical performance values must be obtained from the student's own execution.

31. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Diederik P. Kingma and Max Welling, "Auto-Encoding Variational Bayes," ICLR, 2014.
3. Pascal Vincent et al., "Stacked Denoising Autoencoders: Learning Useful Representations in a Deep Network with a Local Denoising Criterion," Journal of Machine Learning Research, 2010.

4. Yann LeCun, Corinna Cortes and Christopher J. C. Burges, “MNIST Handwritten Digit Database.”
5. TensorFlow Documentation: <https://www.tensorflow.org>
6. Keras Documentation: <https://keras.io>
7. scikit-image Documentation: <https://scikit-image.org>